Chisel-segmentation pipeline: beginner handbook

ClearML + Hydra + Kaggle. One model per class for the Chisel server.

Generated from docs/GUIDE.md, 2026-11-10

Contents

Part 1. Orientation

1.1 The problem
1.2 The big picture
1.3 Why it is built this way (the decisions)
1.4 ClearML in ten minutes
1.5 What each folder does (a tour)
1.6 Which file runs first, and which next
1.7 A suggested learning path

Part 2. Doing it step by step

2.1 One-time setup on your computer
2.2 The data
2.3 A first run on your own computer
2.4 Base tasks: the templates every remote run copies
2.5 The Kaggle agent
2.6 One remote training run
2.7 The per-class pipeline

Part 3. How the code works

3.1 Follow one training run (`src/training/train.py`)
3.2 How settings travel: Hydra, Overrides and recompose (`src/common/clearml_utils.py`)
3.3 The data gate (`src/gates/domain_check.py`, `src/gates/preflight.py`)
3.4 The serving gate (`src/gates/serve_check.py`, `src/gates/serve_gate.py`)
3.5 The pipeline code (`src/orchestration/pipeline.py`, `pipeline/pipeline.yaml`)
3.6 The Kaggle agent (`kaggle/start_agent.py`)

Part 4. From checkpoint to production

4.1 Export (`src/export_onnx.py`)
4.2 What the production server does (Chisel, read from its code)
4.3 The one change the server needs
4.4 What the gates measured so far

Part 5. Reference

5.1 Recipes
5.2 Troubleshooting (every row happened in this project)
5.3 Production monitoring
5.4 Going offline (Untested, except the weights)
5.5 What is proven and what is not
5.6 Glossary
5.7 Cheat sheet

Part 6. Code reference (annotated excerpts)

6.1 Start-up: the config helper every command loads first
6.2 ClearML <-> Hydra: how a run gets its settings (`init_clearml`)
6.3 Turning ClearML values back into Hydra overrides (`_fmt`, `_quote`, `recompose`)
6.4 Failing a gate (`fail_gate`)

6.5 The dataset: where the data comes from and what a sample is (`src/data/tiles.py`)
6.6 One training run, top to bottom (`src/training/train.py`)
6.7 Loss and metrics (`src/training/module.py`)
6.8 Choosing the threshold: the sweep (`src/training/sweep.py`)
6.9 The data gate (`src/gates/preflight.py`)
6.10 The serving measurement (`src/gates/serve_check.py`)
6.11 The serving gate as a task (`src/gates/serve_gate.py`)
6.12 The pipeline (`src/orchestration/pipeline.py`)
6.13 ONNX export (`src/export_onnx.py`)
6.14 Making the file smaller (`src/shrink.py`)
6.15 Building the network (`src/model_factory.py`)
6.16 Inside the server: probabilities -> mask (`serving/postprocess_semantic.py`)
6.17 The server change (`serving/chisel_server.patch`)
6.18 Turning a Kaggle notebook into an agent (`kaggle/start_agent.py`)
6.19 Which settings ClearML may change (`conf/config.yaml`)
6.20 One architecture per class (`pipeline/pipeline.yaml`)

This handbook is for someone who has never used ClearML. It explains **what the project does, why it is built this way, how to run every part step by step, and how the code flows**. Read Part 1 first; use Parts 2 to 4 as you do the work; keep Part 5 for when something breaks.

Status labels used throughout:

- **Verified**: run end to end during development (on an M3 Mac and/or a Kaggle Tesla T4).
- **Untested**: written but never run. Treat it as a draft.

Contents

1. Orientation: the problem, the big picture, the ideas behind the design, and the order in which files run
2. Doing it step by step (setup, data, local run, base tasks, Kaggle agent, one remote run, the pipeline)
3. How the code works (follow one run through the files)
4. From checkpoint to production (ONNX, the server, the gates)
5. Reference: recipes, troubleshooting, offline, glossary, cheat sheet, what is and is not proven
6. Code reference: the important pieces of the code, with what each one does (at the very end)

Part 1. Orientation

1.1 The problem

Aerial image crops come in from users. For each crop, a production service called **Chisel** answers: where are the **buildings, roads, water and vegetation**? It does this with **one model per class**, four models in one container (limits: 4 vCPU request, 8 vCPU limit, 8 GiB memory), on tiles of 1280 x 1280 pixels.

This repository is the **training side**: it trains one model per class, checks that the models will fit in the container *before* spending GPU time on them, and exports them in the form the service loads.

The classes are learned from the public **LoveDA** dataset, whose masks hold class ids: `0 no-data, 1 background, 2 building, 3 road, 4 water, 5 barren, 6 forest, 7 agriculture`.

1.2 The big picture

The whole project is one flow, from raw images to models running in the server. Each stage lives in its own folder (the folder is in brackets).



In words:

1. **Data.** You turn LoveDA into a train/val split once and upload it to ClearML. Remote machines download it by id.
2. **Pipeline.** You describe *what* to run. The pipeline script creates one ClearML task per step and queues them.
3. **Gates and training.** An agent on a GPU machine takes the tasks one by one. The two gates run first; if either fails (exit code 2), training is never started. Then one model per class is trained, and a last gate checks that the four real models still fit the container together.
4. **Export.** Every training task ends by writing its ONNX model and the threshold file, as task artifacts.
5. **Serving.** The ONNX files go into the server with a small patch. Every result (metrics, files) is in the ClearML web UI along the way.

1.3 Why it is built this way (the decisions)

Decision	Why
Everything runs as ClearML tasks	Every run is recorded: code version, settings, logs, metrics, files. You can reproduce or compare any run later, and a remote GPU can run it without your laptop.
Hydra holds all settings	One place (`conf/`) defines every setting; any of them can be changed from the command line as `key=value`. ClearML only *supplies override values*; it never edits the YAML. Two config authorities would drift apart.
Only keys in `clearml.tunable` can change remotely	A short, explicit list of safe knobs (epochs, learning rate, class, ...) keeps remote runs predictable. To open a new knob, add it to that list.
Agents run clones of "base tasks", pinned to a git commit	The agent checks out exactly that code, so a run is reproducible. The price: after you push new code you must re-register the base tasks (section 2.4).
Checks ("gates") run before training	A model that can never fit the container, or a dataset that is inconsistent, should fail in minutes, not after hours of GPU time.
One model per class	That is how production serves them. Each class gets its own architecture, threshold and quality number, and can be retrained alone.
Semantic segmentation exported to ONNX	The server already loads ONNX with onnxruntime. Only its post-processing changes (section 4.3).
Training uses the server's input scaling (RGB / 255, no mean/std)	What the model sees in training must equal what it sees in production.
Thresholds are chosen at "no test-time augmentation"	The server does not run TTA, so the threshold that matters is the best one *without* it.
The dataset lives in ClearML	A remote agent has none of your local files. It downloads the dataset by id.
Masks are stored as raw LoveDA class ids	Which ids form "vegetation" (forest + agriculture) is a config choice (`conf/data/*.yaml`), so changing a class never needs a new upload.
Kaggle for the GPU	Free GPU while you build the system. Cost: its agent is fragile (section 2.5). The final home is an offline environment (section 5.4).

1.4 ClearML in ten minutes

ClearML word	Plain meaning	In this project
Server	The website and database that store everything	Hosted: https://app.clear.ml (free account)
Project	A folder of tasks	`chisel-segmentation`
Task	One recorded run of a script: code version, settings, logs, metrics, files	One run of `src.training.train`, `src.gates.domain_check`, ...
Hyperparameters	Settings stored on a task; editable in the UI before a re-run	The `Overrides` section (section 3.2)
Artifact	A file attached to a task	`best_checkpoint`, `metadata`, `onnx`, `onnx_meta`
Model	A checkpoint file ClearML tracks	`best.ckpt` (captured automatically)
Dataset	A versioned folder of data stored on the server	`loveda-multi`
Queue	A waiting list of tasks	`kaggle`

ClearML word	Plain meaning	In this project
Agent / worker	A program that takes tasks from a queue and runs them	`clearml-agent` inside a Kaggle notebook
Draft / clone / enqueue	A task that is not running / a copy of a task / put a draft on a queue	How every remote run is made
Pipeline	Tasks chained so that one starts after another finishes	`gated-training-per-class`

The central idea: **a task stores which git commit, which command and which settings to run.** An agent checks out that commit, installs the packages, applies the settings and runs the command. Your laptop only creates the task and watches it.

1.5 What each folder does (a tour)

Read the folders in this order. For each: what it is for, when you open it, and the files inside.

`conf/`: every setting

What it is. The Hydra configuration, the single source of truth for every number and choice (classes, network, loss, epochs, gate limits). Code never hard-codes a setting; it reads `cfg.<key>`. **You open it when** you want to change a setting for good, or see what a run used. For a one-off change, pass `key=value` on the command line instead.

File / folder	Holds
`config.yaml`	the defaults, training / sweep / export settings, and `clearml.tunable`: the keys ClearML may change remotely
`data/`	which LoveDA mask ids form each class: `building`, `road`, `water`, `vegetation` (one model each), `loveda` (all four in one model), `buildings` (legacy binary masks); `oem_building`, `oem_road`, `oem_water`, `oem_vegetation`, `oem` do the same for OpenEarthMap ids (`src/data/prepare_oem.py`)
`architecture/`, `encoder/`	the network (Unet, FPN, DeepLabV3+) and its encoder and pretrained weights
`loss/`, `preflight/`, `serve/`	loss weights; the data-gate limits; the serving-gate limits (the production container)

`src/data/`: getting the data ready

What it is. Everything about images and masks. **You open it when** the data changes or a run cannot find it.

File	Does
`prepare.py`	LoveDA flat folder -> `train/` and `val/` (random split per tile, seeded)
`upload.py`	a train/val folder -> a versioned ClearML Dataset; prints its id
`tiles.py`	the dataset a run reads: image -> tensor (RGB / 255, like the server), mask -> one binary channel per class

`src/training/`: one training run

What it is. Section 3.1 follows a run through these files. **You open it when** you change how a model learns or what a run records.

File	Does
`train.py`	the entry point: data -> training -> sweep -> metadata -> ONNX -> artifacts
`module.py`	the LightningModule: BCE + Dice loss, IoU and F1 per class
`sweep.py`	after training: every test-time augmentation x threshold, to choose the threshold per class
`metadata.py`	`metadata.json`: config, package versions, checkpoint hash, sweep results
`logger.py`	sends Lightning's metrics to ClearML

`src/gates/`: stop early when something cannot work

What it is. Checks that fail their task (exit code 2) so a pipeline stops before wasting GPU time. **You open it when** a gate fails, or you want to change what it checks (the limits themselves are in `conf/`).

File	Does
`domain_check.py`	the data gate as a ClearML task
`preflight.py`	the data checks: do train and val look alike (domain shift)? same class shares (label balance)?
`serve_gate.py`	the serving gate as a ClearML task, before training (untrained models) and after (real models)
`serve_check.py`	the measurement: all models in one process, with the server's threads and tiles; also a command line tool

`src/orchestration/`: many tasks at once

What it is. Scripts that create and chain ClearML tasks. **You open it when** you add a step or change the order.

File	Does
`pipeline.py`	the per-class pipeline (stage 2 and 3 above); `--dry-run` prints it without touching ClearML
`hpo.py`	Optuna hyperparameter search over clones of the train task (**Untested**)

`src/common/`: shared helpers

File	Does
`clearml_utils.py`	the bridge between Hydra and ClearML: Task.init, the editable "Overrides", recompose, failing a gate (section 3.2)
`envinfo.py`	the container's CPU / memory limits and a process's peak memory

Top of `src/`: export and the network

File	Does
`export_onnx.py`	checkpoint -> `<class>.onnx` (sigmoid output, the server's input format) + `<class>.onnx.json` (threshold)

File	Does
`shrink.py`	stores the ONNX weights as int8: about 4x smaller file, same latency
`model_factory.py`	builds the network with segmentation_models_pytorch and loads the encoder weights
`export_weights.py`	saves pretrained encoder weights to a file, for training without internet
`__init__.py`	registers the `${len:...}` helper the config uses to count classes

`serving/` and `tests/`: the server side

File	Does
`serving/postprocess_semantic.py`	drop-in for the server's YOLO post-processing: probabilities -> 0/255 mask per tile
`serving/chisel_server.patch`	the change to the server's `server.py` (section 4.3)
`tests/test_postprocess_semantic.py`	checks that the post-processing undoes the server's own letterbox exactly

The rest

Folder	Holds	You open it when
`pipeline/pipeline.yaml`	base task names, queues, architecture + encoder per class	you change a class's network or a queue
`kaggle/`	`start_agent.py`: turns a Kaggle GPU notebook into a ClearML agent	every Kaggle session (section 2.5)
`hpo/space.yaml`	the HPO search space	you run `src.orchestration.hpo`
`offline/`	Dockerfile and wheelhouse script for a machine without internet (**Untested**)	you move offline (section 5.4)
`docs/`	this handbook	you are reading it

1.6 Which file runs first, and which next

Three views of the same thing. Pick the one that answers your question.

View A. The order of the commands YOU run (the whole project, start to finish)

#	When	You run	File that starts	What it does
0	once	`pip install -r requirements.txt`, `clearml-init`	-	environment and ClearML credentials
1	once per data version	`python -m src.data.prepare`	`src/data/prepare.py`	raw LoveDA -> `train/` and `val/`
2	once per data version	`python -m src.data.upload`	`src/data/upload.py`	uploads the folder; prints the dataset id

#	When	You run	File that starts	What it does
3	optional, any time	`python -m src.training.train clearml.enabled=false ...`	`src/training/train.py`	smoke test on your own computer
4	after every `git push`	`python -m src.gates.domain_check`, `python -m src.gates.serve_gate`, `python -m src.training.train` (no arguments, stop after ~80 s)	those three files	registers the **base tasks** on the new commit
5	every Kaggle session	run the notebook cells	`kaggle/start_agent.py`	starts the agent that will run the tasks
6	each experiment	`python -m src.orchestration.pipeline ...`	`src/orchestration/pipeline.py`	creates the tasks and puts them on the queue
7	after the pipeline	(the agent works; you watch the web UI)	-	metrics, models and ONNX files appear as task artifacts
8	when needed	`python -m src.export_onnx`, `python -m src.gates.serve_check`, `python tests/test_postprocess_semantic.py`	those files	export or measure by hand; the server integration is yours to do

View B. What the PIPELINE runs, in order (each box is a different script on the agent)

```
your computer:   src/orchestration/pipeline.py  --(creates tasks, puts them on the queue)-->  ClearML queue "kaggl
                                                                                              |
the Kaggle agent takes them (one at a time) and runs, from the repo at the pinned commit:
                                                                                              v
   1  domain_check       -> runs  src/gates/domain_check.py          (data gate)
   2  serve_gate_pre     -> runs  src/gates/serve_gate.py            (untrained models fit together?)
        [1 and 2 run in whichever order the agent picks them; BOTH must pass]
                                                                                              v
   3  train_building  \
      train_road        |-> each runs  src/training/train.py            (one class each, one after another)
      train_water       |
      train_vegetation /
                                                                                              v
   4  serve_gate_post    -> runs  src/gates/serve_gate.py            (the real ONNX models fit together?)
```

If any step fails, the steps after it are never created.

View C. The order of the FILES INSIDE one run

Any `python -m src.<name>` first runs `src/__init__.py` (it registers the `${len:...}` resolver used in the config), then the script's own file. Then:

`src/training/train.py` (3 and the training steps above):

```
src/__init__.py                     registers the resolver
src/training/train.py               module loads, importing the helpers below
   Hydra loads conf/config.yaml + conf/data/<class>.yaml + conf/architecture + conf/encoder -> cfg
   main(cfg) then calls, in this order:
   1  clearml_utils.init_clearml()        Task.init, "Overrides", recompose        (src/common/clearml_utils.py)
   2  tiles.resolve_root()                the ClearML dataset or the local folder   (src/data/tiles.py)
   3  preflight.gate_preflight()          domain-shift / label-balance gate         (src/gates/preflight.py)
        skipped inside the pipeline (preflight.enabled=false); a failure -> exit code 2
   4  tiles.TileDataset (train, val)      images -> tensors, masks -> one channel per class
   5  module.SegModule()                  -> model_factory.build_model()            (src/training/module.py, src/m
   6  trainer.fit()                       each epoch: logger.ClearMLLogger sends metrics   (src/training/logger.py
   7  sweep.run_sweep() / report_sweep()  best TTA and threshold per class         (src/training/sweep.py)
   8  metadata.write_metadata()           writes metadata.json                      (src/training/metadata.py)
   9  export_onnx.export_model()          <class>.onnx + .onnx.json                 (src/export_onnx.py)
  10  upload artifacts, publish the task
```

`src/gates/domain_check.py`: `clearml_utils.init_clearml` -> `tiles.resolve_root` -> `preflight.gate_preflight` (samples train and val with `tiles.TileDataset`, runs the checks) -> exit code 2 if a gate fails.

`src/gates/serve_gate.py` (the two serving gates):

```
src/__init__.py
src/gates/serve_gate.py
  1  clearml_utils.init_clearml()
  2  get the models, depending on serve.source:
       random: export_onnx.export_untrained() for each class
                 -> export_onnx.compose_config()  -> model_factory.build_model()  -> export_onnx.export_model()
       tasks:  download the onnx artifacts of the train_<class> steps (found through the "pipe: <id>" tag)
  3  measure_isolated() starts a NEW process:  python -m src.gates.serve_check
        src/__init__.py -> src/gates/serve_check.py: measure()  (uses serving/postprocess_semantic.py per tile
        and envinfo.peak_rss_gib for memory)  ->  evaluate()  -> writes the result json
  4  back in serve_gate: report to ClearML; on a violation -> clearml_utils.fail_gate(): mark failed, exit code 2
```

`src/orchestration/pipeline.py` (your computer): `plan()` builds the list of steps (a plain list) -> `build()` looks up the newest non-archived base task of each name and hands the steps to ClearML's `PipelineController` -> `start_locally()` or `start()`. After that the controller only watches.

1.7 A suggested learning path

1. Read this Part 1: 1.2 (the five stages), 1.5 (the folder tour), 1.6 (the order files run in). Then section 3.1 (one run through the code) with `src/training/train.py` open next to it.
2. Do section 2.1 to 2.3 (setup, data, a local smoke run). No ClearML server needed for the smoke run.
3. Read 2.4 and 2.5, then try 2.6 (one remote run on Kaggle).
4. Finish with 2.7 (the pipeline) and Part 4 (export, the server, the gates' numbers).

If you want to...	Start in
change a setting	`conf/` (or `key=value` on the command line)
understand one training run	`src/training/train.py`, section 3.1
know why a run was stopped	`src/gates/`, the failed task's console in ClearML, section 5.2
change the pipeline's steps	`src/orchestration/pipeline.py`, `pipeline/pipeline.yaml`, section 3.5
put models into the server	`serving/`, section 4.3

Part 2. Doing it step by step

2.1 One-time setup on your computer

1. Install **Python 3.12**. (The newest Python often has no prebuilt wheels for torch, rasterio or torchgeo.)
2. Create an environment and install the packages:

```
cd chisel_pipeline_new
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

3. Create a free account at https://app.clear.ml. Go to **Settings → Workspace → Create new credentials** and copy the block it shows.
4. Run `clearml-init` and paste the block. It writes `~/clearml.conf`. Check that the first lines say `api.clear.ml`, not `localhost`.
5. Never commit `clearml.conf` or paste its keys anywhere. `.gitignore` already excludes it.

> **Why these pins in `requirements.txt`?** `onnx<1.19` and `protobuf<6`: newer onnx needs protobuf 6, which clashes with Kaggle's preinstalled protobuf 5 (section 5.2). `opencv-python-headless==4.13.0.92`: the version the Chisel server locks; 4.14 crashes on a tiny image resize on Apple Silicon.

2.2 The data

```
python -m src.data.prepare --src data/loveda --out data/loveda_multi   # class-id masks (default)
python -m src.data.upload --path data/loveda_multi --name loveda-multi     # upload; prints: dataset id: <32 hex>
```

`src.data.prepare` copies the images and masks into `train/` and `val/` (80/20, random per tile, seed 42), each with `images/` and `masks/`. The masks keep their original class ids. `dataset.py` uploads the folder as a ClearML Dataset (create, add files, upload, finalize) and prints its id. A run uses the dataset when `data.clearml_dataset_id` is set; a local run uses the folder `data.root`.

> **Why keep raw class ids?** The mapping from ids to classes lives in `conf/data/<class>.yaml`, e.g. `vegetation: [6, 7]`. Regrouping classes never needs a new upload.

Known limit: the split is random per tile, so neighbouring tiles of one scene can land on both sides and make validation scores slightly optimistic. Split by scene/city if you have that information.

A guard in `src/data/tiles.py` stops a run if the class list is set but the masks are the old binary 0/255 ones, instead of silently training on empty targets. Datasets named `loveda-buildings` are binary and only work with `data=buildings`.

2.3 A first run on your own computer

```
python -m src.training.train clearml.enabled=false data=water data.root=data/loveda_multi \
    train.epochs=1 train.num_workers=0 train.batch_size=4 sweep.enabled=false
```

Verified. About 7 minutes per epoch on an M3 Mac, and about 10 GB of memory at batch size 4 (full 1024 x 1024 tiles), so close heavy apps. Settings are Hydra overrides, `key=value`. In zsh, quote brackets: `'sweep.tta=[none]'`. With ClearML on (leave out `clearml.enabled=false`) a task also appears on app.clear.ml.

`data=water` selects `conf/data/water.yaml`: a model with one output channel for water. `data=loveda` trains all four classes in one model; that is **not** the production shape, but the same code does it.

2.4 Base tasks: the templates every remote run copies

Agents and pipelines never run your laptop's process. They **clone a base task**. Three are used, in project `chisel-segmentation`:

Name	Registered by running (no arguments!)	Used for
`domain-check`	`python -m src.gates.domain_check`	the data gate
`serve-gate`	`python -m src.gates.serve_gate`	both serving gates
`Unet-resnet34`	`python -m src.training.train`	the training step (every class)

Registering means: run the script once; ClearML records the repo, the commit, the command, the settings and the required packages. You can stop the run after about 80 seconds: everything is recorded at the start. The base task's own status does not matter, only that it exists.

Rules that cost hours when ignored:

1. **Push first.** The agent clones the code from GitHub (https://github.com/CheeKuang/clearml-kaggle, public). Uncommitted local changes do not exist for it.
2. **A base task is pinned to one commit.** After you push new code, existing base tasks still run the *old* code. Archive the old base task and register again (or, for a draft, `Task.get_task(...).set_script(commit="<sha>")`).
3. **Never register with extra command-line arguments.** ClearML stores them in the entry point *and* in a parameter `Args/overrides`, and every clone inherits them. A smoke test with `sweep.enabled=false` once silently disabled the sweep in later runs. Check: `Task.get_task(...).get_parameters().get("Args/overrides")` should be `None` or `['hydra.mode=RUN']`; remove a stale one with `task.delete_parameter("Args/overrides")`.
4. `Task.set_parameters(...)` **replaces all parameters**; use `update_parameters(...)` to change a few.
5. Find a task by exact name: `Task.get_task(project_name="chisel-segmentation", task_name="serve-gate")`.

> **Why this indirection?** It is what makes a run reproducible and remotely executable, and what lets a pipeline or an HPO job create hundreds of runs from one template.

2.5 The Kaggle agent

The agent is the program that actually runs tasks. Start it in a Kaggle notebook **every session**:

1. New notebook. Settings: **Accelerator = GPU**, **Internet = On**.
2. Add-ons → Secrets: create `CLEARML_KEY` (access key) and `CLEARML_SECRET` (secret key) from your app.clear.ml credentials and attach both to the notebook.
3. Cell 1:

```
!rm -rf /kaggle/working/repo && git clone https://github.com/CheeKuang/clearml-kaggle.git
/kaggle/working/repo
```

4. Cell 2, **last** cell (it keeps running):

```
%run /kaggle/working/repo/kaggle/start_agent.py
```

5. In ClearML (Workers & Queues) a worker such as `8aacb9ba6e24:gpu0` should show up on queue `kaggle`.

The agent dies when the Kaggle session ends (tab closed, idle timeout, time limit); queued tasks then wait forever. Keep the tab open. After a restart the worker has a new name and a cold package cache, so the first tasks are slower (about 3 to 6 extra minutes).

> **Why this is fragile, and the way out.** An interactive notebook is a poor place for a long-lived service. It is fine while building. In production the agent runs on a machine you control (section 5.4).

2.6 One remote training run

```
from clearml import Task
base = Task.get_task(project_name="chisel-segmentation", task_name="Unet-resnet34")
t = Task.clone(source_task=base, name="road-test", parent=base.id)
t.update_parameters({
    "Overrides/data": "road",                                  # which class (conf/data/road.yaml)
    "Overrides/data.clearml_dataset_id": "<dataset id>",
    "Overrides/train.epochs": 30,
    "Overrides/train.batch_size": 4,
})
Task.enqueue(t, queue_name="kaggle")
print(t.get_output_log_web_page())
```

Or in the web UI: open the task → **Configuration** → **Hyperparameters** → **Overrides**, edit, right-click → Enqueue. Timings seen on a Kaggle T4, batch size 4 (the whole `train_water` step with 1 epoch, sweep and export took 11.7 min):

Stage	Time
Package install (cold cache)	3 to 6 min

Stage	Time
Dataset download (2.25 GiB)	about 20 s
One epoch	about 2.5 min
TTA x threshold sweep	about 5 min
ONNX export	about 1 min

A 30-epoch run should take about 75 minutes plus sweep and export (an estimate; **Untested**, only 1 and 2 epochs have been run).

2.7 The per-class pipeline

```
python -m src.orchestration.pipeline --dry-run                    # print every step and its overrides,
touches nothing
python -m src.orchestration.pipeline --local --queue kaggle \
    --param data.clearml_dataset_id=<dataset id> \
    --train-param train.epochs=1 --train-param train.batch_size=4
python -m src.orchestration.pipeline --local --classes road,water ...   # only some classes
```

`--local` runs the controller (the program that creates the steps) on your computer; without it the controller needs an agent on a `services` queue (**Untested**, not set up). Options: `--queue Q`, `--classes a,b`, `--param k=v` (every step), `--train-param k=v`, `--serve-param k=v`, `--dry-run`, `--config path`. The pipeline refuses to start until a dataset id is given (`pipeline.yaml` holds `null` as a reminder).

What happens (the order is set by dependencies; one agent runs one task at a time):

1. `domain_check`: are train and val consistent? And `serve_gate_pre`: can untrained models of your chosen architectures run **together** inside the container limits? Both must pass.
2. One `train_<class>` step per class (own architecture and encoder). Each trains, runs the sweep and **exports its ONNX model** with a threshold.
3. `serve_gate_post`: the same measurement on the **real** exported models.
4. A failing gate exits with code 2 and fails its step; nothing after it is created.

Where to look: the **Pipelines** page. Steps are tagged `pipe: <controller id>`.

Verified on Kaggle (2 classes, 1 epoch each, about 33 minutes): every step completed; `train_water` val IoU 0.70, `train_road` 0.48 (one epoch only, so these say "training works", not "the models are good"); `serve_gate_post` found both models by the pipeline tag and passed. A separate gate run with **all four** untrained models: 1.74 GiB peak, 8.6 s per request, 3.8 cores busy, on a 4-CPU container.

Part 3. How the code works

3.1 Follow one training run (`src/training/train.py`)

```
python -m src.training.train data=road ...
   |
   1  Hydra reads conf/config.yaml + the chosen groups (data=road, architecture=unet, ...) + your overrides -> cfg
   2  init_clearml(cfg)            Task.init, publish "Overrides", apply remote overrides, store the resolved conf
   3  seed_everything              same seed -> repeatable (train.deterministic: deterministic kernels too)
   4  resolve_root(cfg)            the ClearML Dataset (downloaded to a cache) or the local folder data.root
   5  preflight (if enabled)       domain shift + label balance; a failed `gate` -> task failed, exit code 2
   6  fingerprints                 sha256 of every data file, split.json, sha256 of the starting weights -> "Run"
   7  Lightning training           SegModule on TileDataset; keep the best checkpoint by val/iou; early stopping
   8  load the best checkpoint
   9  sweep (if enabled)           every TTA x threshold on val -> best pair
  10  ONNX export (if enabled)     <class>.onnx + .onnx.json (threshold at TTA "none"), then shrunk/<class>.onnx
                                   (the production file), both checked through the server's pre/post-processing
  11  metadata.json; upload artifacts; publish the task (read-only)
```

The pieces behind each step:

- **Data** (`src/data/tiles.py`). `TileDataset` reads an image (rasterio), scales it `/ 255`, and builds the mask as **one binary channel per class**: a pixel is 1 where the mask value is in that class's id list (`np.isin(mask, ids)`). With `data=road` the mask is `(1, H, W)`; with `data=loveda` it is `(4, H, W)`. Training augmentation is random flips and 90-degree rotations, applied to image and mask together; the probabilities are in `augment` (conf/config.yaml).
- **Model** (`src/model_factory.py`, `src/training/module.py`). The network is built from the YAML in `conf/architecture/` and `conf/encoder/` by `segmentation_models_pytorch` (Unet, FPN, DeepLabV3+; ResNet encoders). Its output has `model.classes` channels, computed from the class list (`${len:${data.classes}}`). Pretrained encoder weights come from `smp`, from a local file (`encoder=resnet34_local`) or from torchgeo. The loss is `bce_weight * BCE + dice_weight * Dice`; the Dice loss is `binary` for one class and `multilabel` for several. Validation accumulates true/false positives per class and logs **IoU and F1**; `val/iou` is the mean over classes and is the objective the best checkpoint and HPO follow.
- **Sweep** (`src/training/sweep.py`). On the validation set it tries each test-time augmentation (`none`, `flip`, `d4`) at each threshold from 0.30 to 0.70, records IoU/F1 per class, and picks the best row. This is how a threshold per class is chosen.
- **Metadata** (`src/training/metadata.py`). `metadata.json` freezes the run: resolved config, package versions, GPU, checkpoint hash, the sweep table, the best TTA and threshold (also per class), git status, the hash of every data file, `split.json`, the hashes of the starting weights and of both ONNX files.

What a train Task records, and where to see it in the ClearML UI (enough to rerun it: clone the Task and enqueue):

Tab	What
Execution	git repo, commit, uncommitted diff, installed packages
Hyperparameters -> Overrides	the editable keys (`clearml.tunable`); the only section a clone reads back
Hyperparameters -> Resolved	every final setting, one field each (epochs, lr, loss weights, augment, seed, ...). Display only
Hyperparameters -> Run	tiles and sha256 per split, `split.*` (from `split.json`), steps per epoch, parameters, sha256 of the starting encoder / model weights, scheduler, device, GPU, precision, sha256 of `onnx` and `onnx_shrunk`. Display only
Scalars	`train/loss`, `train/lr`, `val/loss`, `val/iou`, `val/f1` per epoch; sweep IoU; single values `best_val_iou`, `sweep_best_*`, `onnx_iou`, `onnx_shrunk_iou`, `onnx_shrunk_iou_drop`
Plots, Debug samples	sweep table; validation prediction (red) vs ground truth (green)
Artifacts	`resolved_config`, `hydra_snapshot`, `best_checkpoint`, `metadata`, `onnx`, `onnx_shrunk`, `onnx_meta`

A file in production traces back through its `.onnx.json`: `source.task_id` is the train Task, `files.onnx_shrunk.sha256` must equal the sha256 of the deployed file. Verified on the Mac: two runs with the same settings gave bit-identical weights and ONNX files; on a GPU some kernels have no deterministic version (a warning is printed), so expect near-identical rather than bit-identical results there. Datasets made before `split.json` existed show `split = not recorded`; their per-split sha256 still pins the exact files.

> **Why a sweep instead of a fixed 0.5?** Models are rarely best at 0.5, and the optimum differs per class (the server also uses a different confidence per class). Doing it once after training makes the choice evidence-based.

3.2 How settings travel: Hydra, Overrides and recompose (`src/common/clearml_utils.py`)

`init_clearml(cfg)` is the bridge between Hydra and ClearML:

```
Hydra config (local YAML + CLI)                          ClearML task on the server
        |                                                          |
        |  1  Task.init  ------------------------------------------> task created (repo, commit, command)
        |  2  state = {tunable key: value as TEXT}                 |
        |     task.connect(state, name="Overrides")  ----------> shows as editable "Overrides"
        |                                                          |
        |  <------ on an AGENT: returns the values from the task (possibly edited in the UI / by a pipeline)
        |  3  if they differ from the local ones: recompose = rebuild the Hydra config with them
        |  4  task.connect_configuration(final config) -------> stored for the record
```

Remember:

- Only keys in `clearml.tunable` (in `conf/config.yaml`) are connected. `data`, `architecture` and `encoder` are **config groups**; they are allowed as keys and switch whole YAML files.

- Everything is connected **as text**. ClearML otherwise casts a remote value back to the type of the local default (it silently dropped `0.3` over an int `8`); Hydra parses the text when the config is recomposed. Values that contain `= , : [ ] { } ( )` or spaces are quoted for Hydra (`_quote`).
- `connect()` updates the dict you pass **in place** on remote runs, so the code keeps a copy of the original before calling it. Comparing the dict with itself once meant remote overrides were never applied.
- On a local run nothing differs, so nothing is recomposed: local behaviour is plain Hydra.

> **Why this design?** The alternative (ClearML editing a copy of the config) gives two sources of truth. Here the final config is always a Hydra config, so a run is the same whether started by you, a pipeline or HPO.

3.3 The data gate (`src/gates/domain_check.py`, `src/gates/preflight.py`)

It builds a sample (up to 500 tiles) of the train and val splits and runs two checks:

- **Domain shift**: summarise every image by per-channel mean/std and percentiles, then ask how often a tile's nearest neighbours come from the *other* split. Score 0 means perfectly mixed, 1 means fully separated. Gate: fail above 0.5. Measured: 0.004.
- **Label balance**: for every class, compare the share of positive pixels per tile between splits (Wasserstein distance and mean-ratio gap). Reports only (diagnostic), per class.

> **Why a data gate?** A train/val split that looks different means validation scores lie. Catching that costs minutes; finding out after a 75-minute run costs a night.

3.4 The serving gate (`src/gates/serve_check.py`, `src/gates/serve_gate.py`)

The question: *can all four production models run together inside the container?* The check copies how the Chisel server runs inference: onnxruntime CPU sessions (2 threads each, memory arena off), a thread pool of 2, a lock per model, tiles of 1280 x 1280. It measures the time of a request and the peak memory, and counts how many CPU cores were busy.

`serve_gate.py` wraps it as a ClearML task with three sources of models (`serve.source`):

Source	Models	When
`random`	untrained models of the architectures in `serve.spec` (exported to ONNX on the spot)	before training. A random model costs the same to run as a trained one, so no training is needed to know if it fits.
`tasks`	the `onnx` artifacts of the sibling `train_<class>` steps (found via the `pipe: <id>` tag)	after training
`folder`	the `.onnx` files in `serve.folder`	local use

> **Why measure in a separate process?** Exporting a model uses a lot of memory. If the measurement ran in the same process, that peak would be counted (and on Linux a child process even inherits its parent's counter). So the gate exports first, then starts a fresh process and reads *that* process's own peak (`VmHWM`).

> **Important:** the numbers mean something only inside a container that enforces the real limits (a Kubernetes pod or `docker --cpus=8 --memory=8g`). On Kaggle (4 CPUs, 30 GB) the gate says so in its warnings. Not covered: image decoding, alpha masking, winner-take-all, colouring and PNG encoding of the real server.

3.5 The pipeline code (`src/orchestration/pipeline.py`, `pipeline/pipeline.yaml`)

`plan(config)` returns a plain list of steps, each a dict of ClearML `add_step` arguments: name, parents, base task, queue and `parameter_override` (the "Overrides/..." values). `build()` feeds them to a ClearML `PipelineController`. Because the plan is plain data, `--dry-run` can print it without touching ClearML.

```
domain_check            Overrides: data=loveda, data.clearml_dataset_id
serve_gate_pre          Overrides: serve.source=random, serve.spec="building=unet:resnet34_imagenet,road=..."
train_<class> (each)    parents: domain_check, serve_gate_pre
                        Overrides: data=<class>, architecture, encoder, preflight.enabled=false, train params
serve_gate_post         parents: all train_<class>;  Overrides: serve.source=tasks
```

`pipeline.yaml` has one `classes:` table, the single source for each class's architecture and encoder. The same table builds the `serve.spec` text for the pre-gate and the train overrides, so the gate always measures what is trained.

> **Why is `preflight.enabled=false` on the train steps?** `domain_check` already gated the run; repeating it would only cost time.

3.6 The Kaggle agent (`kaggle/start_agent.py`)

It reads the two secrets, sets the ClearML host variables, tells the agent to reuse Kaggle's preinstalled packages (`system_site_packages`), installs `clearml-agent` and runs `clearml-agent daemon --queue kaggle --gpus 0`. For each task the agent: checks out the pinned commit, builds a virtual environment from the task's recorded requirements, runs the command, and streams logs back.

Part 4. From checkpoint to production

4.1 Export (`src/export_onnx.py`)

At the end of every training run, `export_model` wraps the network with a **sigmoid** and writes `<class>.onnx`:

- input `images`: float32 `(1, 3, 1280, 1280)`, RGB scaled to 0..1 (exactly the server's preprocessing),
- output `probs`: float32 `(1, C, 1280, 1280)`, probabilities.

It then loads the file with onnxruntime and checks it reproduces PyTorch (max difference about 1e-5, **Verified**). A sidecar `<class>.onnx.json` carries the class name and the **threshold chosen at "no TTA"**. Both are uploaded as the task artifacts `onnx` and `onnx_meta`.

4.2 What the production server does (Chisel, read from its code)

- It receives a base64 image (RGBA; the alpha channel marks the visible area) and a total area, and returns, per category, an area and a coloured PNG mask, plus an `open` category for unclaimed visible pixels.
- Images larger than 1280 are cut into 1280 x 1280 tiles (edges padded); each tile goes through all four models in a thread pool of 2, each model's onnxruntime session using 2 threads.
- Each class has its own confidence. Overlaps are resolved **winner-take-all**: masks are 0/255, so ties go to the first category in the settings order (vegetation, building, water, road).
- The **current** models are YOLO instance-segmentation models. Ours are **semantic**.

4.3 The one change the server needs

`serving/postprocess_semantic.py` is a drop-in for the server's `postprocess_yolo_masks` (same signature, same 0/255 result). It removes the letterbox padding, resizes the probabilities back to the tile and thresholds them. The rest of the server (slicing, thread pool, alpha mask, winner-take-all, colours, areas) stays unchanged. Its geometry is tested against the server's own `letterbox` (`tests/test_postprocess_semantic.py`: IoU 1.0000 on six tile shapes, **Verified**).

`serving/chisel_server.patch` is the change to the server's `server.py`: a model with a `<name>.onnx.json` sidecar uses the sidecar's threshold and `postprocess_semantic_masks`; a model without one keeps the YOLO path and its `confidence`. Apply it to a COPY of the server, copy `serving/postprocess_semantic.py` into its `libs/`, and put the four `.onnx` + `.onnx.json` files in its models folder. **Verified on a copy** (onnxruntime 1.28.0 as in the server lock file, the server's `predict()` called directly, no HTTP), with the 1-epoch models on RGBA images cut from LoveDA val:

Image	IoU vegetation / water / building / road
2048 x 2048, 4 tiles, polygon alpha	0.82 / 0.60 / 0.53 / 0.22
1500 x 900, opaque	0.82 / 0.57 / - / - (no building or road in it)
400 x 400 crop, circular alpha	vegetation 0.00 (71% of the area is vegetation)

Small crops fail. The server letterboxes a crop up to 1280, and the models were trained at LoveDA's scale. On one tile cropped to 1024 / 800 / 640 / 400 px (upscaled 1.2 / 1.6 / 2.0 / 3.2 times), water IoU was 0.80 / 0.76 / 0.67 / 0.07. Options: train with random up-scaling, pad small crops instead of enlarging them (a server change), or set a minimum crop size. **Untested:** a real production image, the HTTP path through litserve.

Be aware: the server returns **empty masks without an error** if a model fails, times out, or the image exceeds its pixel limit. A passing gate does not rule that out.

4.4 What the gates measured so far

Setup	One 1280 x 1280 tile, four models	Peak memory
Current YOLO models, M3 Mac, noise input	5.97 s	4.1 GiB
Untrained Unet-resnet34 per class, M3 Mac	4.4 s	2.8 GiB
Untrained Unet-resnet34 per class, Kaggle 4-CPU container	8.6 s	1.74 GiB

The machines differ, so the rows are not comparable to each other; they only show the order of magnitude and that memory is far below 8 GiB. The server's pool of 2 keeps only about 4 cores busy; a pool of 4 used 8 cores and was about 1.4 times faster at about 2 GiB more memory (Mac). Whether the latency suits you depends on a target I do not have.

Under docker limits (`--cpus 4|8 --memory 8g`, the server's own `predict()` in a container with its pinned packages, one request at a time, Docker Desktop VM on the M3, so not the production hardware):

Request	CPUs	Pool	1-epoch Unet-resnet34 (ours)	Current YOLO
1 tile	4	2	4.6 s, 1.86 GiB	6.5 s, 2.76 GiB
1 tile	4	4	6.6 s, 2.86 GiB	9.6 s, 3.99 GiB
1 tile	8	2	4.6 s, 1.86 GiB	6.5 s, 2.62 GiB
1 tile	8	4	3.4 s, 2.86 GiB	4.9 s, 4.01 GiB
4 tiles (2048 x 2048)	4	2	18.2 s, 1.96 GiB	26.1 s, 2.81 GiB

Memory is the container's cgroup peak. Keep the pool at 2 on 4 CPUs (pool 4 is slower there); pool 4 helps only when the 8-CPU limit is really available.

Part 5. Reference

5.1 Recipes

I want to...	Do this
Try another architecture or encoder for a class	Add a YAML in `conf/architecture/` or `conf/encoder/`; set it for that class in `pipeline/pipeline.yaml` (`classes:`), or pass `architecture=` / `encoder=`. `resnet50_torchgeo_s2` needs 13-channel Sentinel-2 data; LoveDA tiles have 3.
Change a training setting	Edit `conf/config.yaml` or pass `key=value`. For a remote run the key must be in `tunable`.
Change what a class means	Edit the id list in `conf/data/<class>.yaml`. No code change; the data must contain those ids.
Add a class	Add `conf/data/<name>.yaml` (copy one), add it to `classes:` in `pipeline.yaml`, and to the server's settings.
Use a new dataset version	`python -m src.data.upload --path <folder> --name loveda-multi --parent <old id>`, then use the new id.
Make a new setting editable remotely	Add its key to `clearml.tunable` in `conf/config.yaml`; push; re-register base tasks.
Export a model by hand	`python -m src.export_onnx --checkpoint best.ckpt --metadata metadata.json --out models/road.onnx`
Measure models by hand	`python -m src.gates.serve_check --models models --tiles 1`
Search hyperparameters	Edit `hpo/space.yaml`, `python -m src.orchestration.hpo --mode clearml`. **Untested** here.
Compare architectures per class	Run the pipeline once per setting, or clone a train step; compare `val/iou_<class>`.

After any code change: push, then re-register or re-pin the base tasks (section 2.4).

5.2 Troubleshooting (every row happened in this project)

Symptom	Cause	Fix
`No images under data/...` on the agent	Remote overrides were not applied (a comparison bug)	Fixed in `clearml_utils.py`; make sure the base task points at a commit that has the fix.
A float override is ignored (`0.3` stays `8`)	ClearML casts remote values to the default's type	Fixed: overrides are connected as text.
A pipeline step ran with `sweep.enabled=false`	Stale `Args/overrides` on the base task	Section 2.4, rule 3.
Task hangs at `import triton`, memory flat	`segmentation_models_pytorch` imported after ClearML's import hooks	Import it at the top of the script, before `Task.init` (`src/gates/serve_gate.py`; `src/training/train.py` imports it via `module.py`).
`Protobuf Gencode/Runtime ... mismatched` when exporting on Kaggle	`onnx>=1.19` needs protobuf 6; Kaggle's system protobuf is 5, and its `google.*` packages win over a venv copy	Pin `onnx<1.19` and `protobuf<6` (done in `requirements.txt` and `clearml_utils.py`).

Symptom	Cause	Fix
Peak memory reads `0.0`, or is far too high	`ru_maxrss` is KiB on Linux, bytes on macOS, and is inherited by child processes	`envinfo.peak_rss_gib` uses `VmHWM` on Linux.
`conf/data` missing in the repo	`.gitignore` had `data/`, which also matched `conf/data/`	Now `/data/`.
Queue has tasks, nothing runs	No worker: the Kaggle session ended	Restart the notebook cells (2.5). Stop stranded tasks and remove stale queue entries.
`BackendError: No user secrets exist ...`	Wrong secret names	`CLEARML_KEY` and `CLEARML_SECRET`, attached to the notebook.
`ValueError: None of the class ids ... occur`	Class list set but masks are binary 0/255	Use `data=buildings`, or a dataset made by `python -m src.data.prepare --mode labels`.
Python segfaults in `cv2.resize` on a Mac	OpenCV 4.14 on Apple Silicon	`opencv-python-headless==4.13.0.92`.
Local training uses 17 GB, the Mac crawls	Full-size tiles, batch 4, other heavy jobs	Lower `train.batch_size`; close apps.
`zsh: no matches found: sweep.tta=[none]`	zsh treats brackets as a pattern	Quote the argument.
A pipeline step's base task is not found	Names must match exactly	`domain-check`, `serve-gate`, `Unet-resnet34`.
Pipeline steps run an OLD commit after re-registering	ClearML's lookup by name includes archived tasks and skips `failed` ones (a base task stopped after ~80 s can show `failed`)	Fixed: `build()` uses the newest non-archived task of each name (`newest_base_task_id`). Check the step's commit in its Execution tab.
`serve_gate_post` never appears, train steps done	The controller (`--local`) ran in a terminal/session that ended	Clone `serve-gate`, set `Overrides/serve.source=tasks` and the dataset id, add the tag `pipe: <controller id>`, enqueue. Keep the controller's terminal open.
An "old" pipeline keeps queuing steps after `pkill`	The controller process survived the signal	Stop it by PID and check with `ps`; mark its controller task and its queued/running steps stopped; check the worker's current task.

Debug trick: test the remote path without a GPU by simulating an agent locally: `Task.debug_simulate_remote_task(task_id=<clone id>)`, then run the script's main.

Before asking for help collect: the task URL, the first error in its console log, the commit it is pinned to, and its `Args/overrides` value.

5.3 Production monitoring

ClearML cannot see model accuracy in production by itself: a deployed model has no ground truth. ClearML Serving can record latency, request counts and custom input/output statistics (for example the predicted building ratio), which are useful early warnings. Accuracy needs a labelled sample scored periodically. None of this exists in the repository yet.

5.4 Going offline (Untested, except the weights)

The target environment has a GPU and no internet. See `offline/README.md`.

Needs internet today	Replacement
ClearML server	Self-hosted ClearML server (docker-compose); change the hosts in `clearml.conf`
Code from GitHub	An internal git server the agent can clone from
`pip install` at task start	A prebuilt image or a package mirror (`offline/Dockerfile`, `offline/build_wheelhouse.sh`)
Pretrained encoder weights (huggingface)	`encoder=resnet34_local` after `python -m src.export_weights`. **Verified**: the model builds with the network off.
Dataset	Re-upload with `src.data.upload` to the internal server
Base tasks	Re-register on the new server

ClearML records the package versions of the machine that registered a task and the agent installs exactly those: the offline image must match them. Task history on the hosted server does not move automatically.

5.5 What is proven and what is not

Area	Status
Per-class pipeline on Kaggle (gates, train, ONNX export, final gate), all 4 classes, 1 epoch	**Verified** (val IoU water 0.70, vegetation 0.59, building 0.53, road 0.48)
Four untrained models together: 1.74 GiB, 8.6 s per tile on a 4-CPU container	**Verified** (not the real envelope)
ONNX equals PyTorch (about 1e-5); post-processing inverts the server's letterbox	**Verified**
Remote overrides, gate failures, config quoting, protobuf pins	**Verified** (found and fixed real bugs)
Model quality after real training (30 epochs, all four classes)	**Untested**: only 1-epoch results exist
The post-processing inside a copy of the server (`serving/chisel_server.patch`), LoveDA RGBA images	**Verified**; a real production image: **Untested**
The server under docker limits (`--cpus 4/8 --memory 8g`), ours vs YOLO	**Verified** on a Mac VM (section 4.4); production hardware / Kubernetes: **Untested**
Small crops (upscaled more than about 2x)	**Fail** (section 4.3)
`src/shrink.py` (int8 weight storage, 97.7 -> 24.6 MB)	file size **Verified**; accuracy of shrunk trained models **Untested**
Controller on a `services` agent, HPO, multi-model comparisons	**Untested**
Offline server / git / package mirror / image	**Untested** (drafts)
Why a train step ends `completed`, not `published`	Not investigated

5.6 Glossary

- **Agent**: program that runs tasks pulled from a queue.
- **Artifact**: a file attached to a task.

- **Base task**: the registered template that every remote run clones.
- **Checkpoint**: saved model weights during training (`best.ckpt`).
- **Class id**: the number in a LoveDA mask that says what a pixel is.
- **Gate**: a check that fails the task (exit code 2) when a limit is violated, so the pipeline stops.
- **Hydra**: the configuration system; `key=value` overrides on the command line.
- **IoU / F1**: overlap scores between predicted and true masks (higher is better, 1 is perfect).
- **Letterbox**: resize an image to fit a square and pad the rest (the server pads with grey 114).
- **Multi-label**: each pixel can belong to several classes; each class is its own yes/no mask.
- **ONNX / onnxruntime**: a portable model file format and the engine that runs it.
- **Overrides**: the editable settings section of a ClearML task.
- **Semantic segmentation**: one prediction per pixel (as opposed to YOLO's separate object instances).
- **Sweep**: trying every test-time augmentation and threshold after training to pick the best.
- **TTA**: test-time augmentation: predict flipped/rotated copies of the image and average them.
- **Threshold**: probability above which a pixel counts as the class.
- **Tunable**: a setting listed in `clearml.tunable`, so it can be changed remotely.

5.7 Cheat sheet

```
# local smoke test, no ClearML
python -m src.training.train clearml.enabled=false data=water data.root=data/loveda_multi train.epochs=1 \
    train.num_workers=0 train.batch_size=4 sweep.enabled=false

# data
python -m src.data.prepare --src data/loveda --out data/loveda_multi
python -m src.data.upload --path data/loveda_multi --name loveda-multi

# register base tasks (after pushing code; NO arguments)
python -m src.gates.domain_check
python -m src.gates.serve_gate
python -m src.training.train            # may be stopped after ~80 s

# the per-class pipeline (a Kaggle agent must be running)
python -m src.orchestration.pipeline --dry-run
python -m src.orchestration.pipeline --local --queue kaggle --param data.clearml_dataset_id=<id> \
    --train-param train.epochs=2 --train-param train.batch_size=4

# production artifacts
python -m src.export_onnx --checkpoint best.ckpt --metadata metadata.json --out models/road.onnx
python -m src.gates.serve_check --models models --tiles 1
python tests/test_postprocess_semantic.py

# hyperparameter search (Untested here)
python -m src.orchestration.hpo --mode clearml
```

Part 6. Code reference (annotated excerpts)

The important pieces of the code, in the order a run uses them. Each excerpt is cut from the file it names, with its line numbers, so it is exactly the code in the repository. Read the explanation under it. Regenerate with `python docs/build_code_reference.py` after changing the code (`docs/build_pdf.sh` does it).

6.1 Start-up: the config helper every command loads first

`src/__init__.py`, lines 1 to 4:

```python
from omegaconf import OmegaConf

# `${len:${data.classes}}` -> number of output channels; 1 when no class list is configured (single binary mask).
OmegaConf.register_new_resolver("len", lambda x: len(x) if x else 1, replace=True)
```

What it does.

Any `python -m src.<something>` imports `src/__init__.py` first. This one line teaches the config a function `len`, so `conf/config.yaml` can say `classes: ${len:${data.classes}}`: the network gets one output channel per class in `conf/data/<class>.yaml` (1 for a per-class model, 4 for `data=loveda`), and 1 when there is no class list.

6.2 ClearML <-> Hydra: how a run gets its settings (`init_clearml`)

`src/common/clearml_utils.py`, lines 111 to 152:

```python
def init_clearml(cfg: DictConfig) -> tuple[Any | None, DictConfig]:
    """Task.init -> connect the tunable keys as "Overrides" -> recompose with remote edits. (None, cfg) when off."""
    if not cfg.clearml.enabled or Task is None:
        return None, cfg
    c, hc = cfg.clearml, HydraConfig.get()
    if untracked := git_state()["untracked"]:
        print(f"[clearml] WARNING: {len(untracked)} file(s) under src/ or conf/ are not in git, so a remote run of this "
              f"Task will not have them (commit and push first): {untracked[:10]}", flush=True)

    # Before Task.init. The agent installs exactly the versions recorded here, so an offline image must match them.
    for pkg in FORCED_REQUIREMENTS:
        Task.add_requirements(pkg)
    # torch.onnx.export imports onnx; onnx >= 1.19 needs protobuf 6 and will not load next to Kaggle's system
    # protobuf 5 (its `google.*` namespace wins over a venv copy). onnx < 1.19 has no such check.
```

```
    Task.add_requirements("onnx", "<1.19")
    Task.add_requirements("protobuf", "<6")

    task = Task.init(project_name=c.project, task_name=c.task_name, task_type=Task.TaskTypes.training,
                     tags=list(c.tags), reuse_last_task_id=False, output_uri=c.output_uri,
                     auto_connect_frameworks={"hydra": False})            # Hydra is handled below (one
authority)

    choices = hc.runtime.choices                                          # config groups use the chosen
option name
    state = {k: _fmt(choices[k] if k in choices else OmegaConf.select(cfg, k)) for k in c.tunable}
    original = dict(state)
    tunables = {k: _fmt(v) for k, v in task.connect(state, name="Overrides").items()}
    # Only the values the clone CHANGED: a clone inherits the base task's value of every key, so re-applying
an
    # unchanged loss.bce_weight=0.5 after loss=road would override road.yaml's own weight.
    if changed := {k: v for k, v in tunables.items() if v != original.get(k)}:
        cfg = recompose(hc, changed)
        print(f"[clearml] config recomposed with remote overrides: {changed}")

    task.connect_configuration(OmegaConf.to_container(cfg, resolve=True), name="resolved_config",
                               description="Final Hydra config after overrides",
ignore_remote_overrides=True)
    # the same final config, one field per value: columns in the experiments table, side by side in Compare
    Reporter(task).params("Resolved", {k: v for k, v in flatten(OmegaConf.to_container(cfg,
resolve=True)).items()
                                       if not k.startswith("clearml.")})
    task.connect_label_enumeration({"background": 0, **{n: i + 1 for i, n in
enumerate(class_names(cfg.data))}})
    task.add_tags([str(cfg.model.arch), str(cfg.model.encoder_name), str(cfg.pretrain.source)])

    if c.execute_remotely:                      # send this run to an agent queue (an agent runs an existing
clone)
        task.execute_remotely(queue_name=c.queue, clone=c.clone_on_remote, exit_process=True)
    return task, cfg
```

What it does.

Every task script calls this first. Step by step:

1. **ClearML off** (`clearml.enabled=false`): return `(None, cfg)`. Every report call then does nothing, so a local run needs no server.
2. `Task.add_requirements`, before `Task.init`: packages ClearML's import scan would miss, plus the `onnx<1.19` / `protobuf<6` pins that keep ONNX export working next to Kaggle's system protobuf 5.
3. `Task.init` creates a new task every time (`reuse_last_task_id=False`), with Hydra's automatic logging off, because Hydra stays the only place settings come from.

4. `state`: the current value of every key in `clearml.tunable`, as text. For a config group such as `data`, the value is the chosen file name (`road`), not its contents.
5. `task.connect(state, name="Overrides")`: on your computer it gives the same values back; on an agent it gives the values the UI or the pipeline wrote into the clone. The code compares against a copy (`original`) because `connect()` changes the dict it is given.
6. **Different?** Then `recompose` builds the Hydra config again with those values (next excerpt).
7. The final config is stored on the task (`connect_configuration`), plus the class names and tags.
8. Only if you asked for it: `execute_remotely` sends this run to a queue.

6.3 Turning ClearML values back into Hydra overrides (`_fmt`, `_quote`, `recompose`)

`src/common/clearml_utils.py`, lines 88 to 94:

```
def _fmt(v: Any) -> str:
    """Value -> the text of a Hydra override (`key=<this>`); also the canonical form for comparisons."""
    if isinstance(v, bool):
        return "true" if v else "false"
    if v is None or v in ("None", ""):
        return "null"
    return str(v)
```

`src/common/clearml_utils.py`, lines 97 to 101:

```
def _quote(text: str) -> str:
    """Hydra override values containing = , : [ ] { } ( ) quotes or spaces must be quoted."""
    if not re.search(r"[=,:\[\]{}()\s'\"\\]", text):
        return text
    return "'" + text.replace("\\", "\\\\").replace("'", "\\'") + "'"
```

`src/common/clearml_utils.py`, lines 104 to 108:

```
def recompose(hc: HydraConfig, tunables: dict[str, Any]) -> DictConfig:
    """Original CLI overrides (minus the tunable keys) + the tunable values, so the edited values win."""
    cli = [o for o in hc.overrides.task if o.lstrip("+~").split("=")[0] not in tunables]
    return hydra.compose(config_name=hc.job.config_name,
                         overrides=cli + [f"{k}={_quote(_fmt(v))}" for k, v in tunables.items()])
```

What it does.

- `_fmt` writes a value the way Hydra expects on a command line: `true`/`false`, and `null` for ClearML's `"None"` or `""`.
- `_quote` puts quotes around values Hydra would otherwise misread (anything with `= , : [ ] { } ( )` or spaces), for example the pipeline's `serve.spec="building=unet:resnet34_imagenet,..."`.
- `recompose` rebuilds the config: the original command-line overrides, minus the tunable keys, plus `key=value` for every tunable key. Dropping them from the command-line part is what lets the edited value win.

6.4 Failing a gate (`fail_gate`)

`src/common/clearml_utils.py`, lines 80 to 85:

```
def fail_gate(task, reason: str, message: str):
    """A gate was violated: mark the Task failed (so a pipeline stops) and exit with code 2."""
    if task:
        task.mark_failed(status_reason=reason, status_message=message[:500])
    print(f"{reason.upper().replace('_', ' ')} FAILED: {message}")
    sys.exit(2)
```

What it does.

Both gates end this way when a limit is broken: the task is marked **failed** (with the reason visible in the ClearML UI), a line such as `SERVE GATE FAILED: ...` is printed, and the process exits with **code 2**. A pipeline step that exits non-zero fails, and ClearML never creates the steps after it.

6.5 The dataset: where the data comes from and what a sample is (`src/data/tiles.py`)

`src/data/tiles.py`, lines 18 to 24:

```
def resolve_root(cfg) -> Path:
    """data.root, or a cached download of the ClearML Dataset data.clearml_dataset_id (~2.25 GiB, ~20 s on Kaggle)."""
    if ds_id := cfg.data.get("clearml_dataset_id"):
        from clearml import Dataset as ClearMLDataset

        return Path(ClearMLDataset.get(dataset_id=ds_id).get_local_copy())
    return Path(cfg.data.root)
```

`src/data/tiles.py`, lines 68 to 72:

```
    def image(self, i) -> np.ndarray:
        x = _read(self.imgs[i], self.c.in_channels).astype(np.float32) / float(self.c.divide_by)
        if self.c.mean is not None:
            x = (x - np.asarray(self.c.mean, np.float32)[:, None, None]) / np.asarray(self.c.std, np.float32)[:, None, None]
        return x
```

`src/data/tiles.py`, lines 74 to 79:

```
    def mask(self, i) -> np.ndarray:
        """(C, H, W) float32, one binary channel per class."""
        m = _read(self.mask_dir / self.imgs[i].name)[0]
```

```
        if self.c.get("classes"):
            return np.stack([np.isin(m, list(ids)) for ids in self.c.classes.values()]).astype(np.float32)
        return (m > 0)[None].astype(np.float32)
```

`src/data/tiles.py`, lines 85 to 95:

```
    def __getitem__(self, i):
        x, y = torch.from_numpy(self.image(i)), torch.from_numpy(self.mask(i))
        if a := self.augment:                    # conf `augment`: random flips + 90-degree rotation, same
for image and mask
            if torch.rand(1) < a.hflip:
                x, y = x.flip(-1), y.flip(-1)
            if torch.rand(1) < a.vflip:
                x, y = x.flip(-2), y.flip(-2)
            if a.rot90:
                k = int(torch.randint(0, 4, (1,)))
                x, y = torch.rot90(x, k, (-2, -1)), torch.rot90(y, k, (-2, -1))
        return x, y
```

What it does.

- `resolve_root`: with `data.clearml_dataset_id` set, the dataset is downloaded once from ClearML into a local cache and that folder is used; otherwise the local `data.root`.
- `image`: reads the file and divides by 255, the same scaling the Chisel server applies (no mean/std).
- `mask`: turns LoveDA's class ids into **one yes/no channel per class**: `np.isin(m, ids)` is 1 where the pixel's id is in that class's list (`vegetation: [6, 7]` -> forest or agriculture). Shape `(classes, H, W)`.
- `__getitem__`: one training sample. With `augment` (training only), random flips and a random 90-degree rotation are applied to image and mask together, so they stay aligned.

6.6 One training run, top to bottom (`src/training/train.py`)

`src/training/train.py`, lines 73 to 133:

```
@hydra.main(config_path="../../conf", config_name="config", version_base="1.3")
def main(cfg: DictConfig):
    out_dir = Path(HydraConfig.get().runtime.output_dir)
    task, cfg = init_clearml(cfg)
    reporter = Reporter(task)
    L.seed_everything(cfg.seed, workers=True)
    OmegaConf.save(cfg, out_dir / "resolved_config.yaml", resolve=True)
    upload(task, resolved_config=out_dir / "resolved_config.yaml", hydra_snapshot=out_dir / ".hydra")

    root = resolve_root(cfg)
    preflight = gate_preflight(cfg, task, reporter, root, out_dir) if cfg.preflight.enabled else {}
    data = data_fingerprint(root, cfg.data.splits.values())                 # which exact files (per-file
in metadata)
```

```
    split = json.loads((root / "split.json").read_text()) if (root / "split.json").exists() else None

    dev = "cuda" if torch.cuda.is_available() else "cpu"          # AMP only on CUDA (Mac MPS runs fp32)
    kw = dict(batch_size=cfg.train.batch_size, num_workers=cfg.train.num_workers, pin_memory=dev == "cuda")
    train_dl = DataLoader(TileDataset(root, cfg.data.splits.train, cfg, augment=True), shuffle=True,
drop_last=True, **kw)
    val_dl = DataLoader(TileDataset(root, cfg.data.splits.val, cfg), **kw)

    module = SegModule(cfg)
    precision = "16-mixed" if cfg.train.amp and dev == "cuda" else 32
    ckpt = ModelCheckpoint(dirpath=out_dir / "checkpoints", filename="best", monitor="val/iou", mode="max",
save_top_k=1)
    trainer = L.Trainer(max_epochs=cfg.train.epochs, accelerator="auto", devices=1, precision=precision,
                        deterministic="warn" if cfg.train.deterministic else None,
                        logger=ClearMLLogger(task) if task else False,
                        callbacks=[ckpt, EarlyStopping("val/iou", mode="max", patience=cfg.train.patience)],
                        default_root_dir=out_dir, enable_progress_bar=False, num_sanity_val_steps=0)
    run = {**{f"{s}_tiles": d["tiles"] for s, d in data.items()}, **{f"{s}_sha256": d["sha256"] for s, d in
data.items()},
           **({f"split.{k}": v for k, v in flatten(split).items()} if split else
              {"split": "not recorded (dataset made before split.json); see the per-split sha256"}),
           "steps_per_epoch": len(train_dl), "model_params": sum(p.numel() for p in module.parameters()),
           "encoder_init_sha256": sha256_state(module.model.encoder.state_dict()),     # the pretrained
starting point
           "model_init_sha256": sha256_state(module.model.state_dict()),
           "scheduler": f"CosineAnnealingLR(T_max={cfg.train.epochs})",
           "device": str(trainer.strategy.root_device), "precision": str(precision),
           "gpu": torch.cuda.get_device_name(0) if torch.cuda.is_available() else None}
    reporter.params("Run", run)
    print(f"[run] {json.dumps(run)}", flush=True)
    trainer.fit(module, train_dl, val_dl)
    module.load_state_dict(torch.load(ckpt.best_model_path, map_location="cpu", weights_only=False)
["state_dict"])
    reporter.single("best_val_iou", module.best_iou)

    df = best = None
    if cfg.sweep.enabled:
        sweep_dl = DataLoader(TileDataset(root, cfg.data.splits[cfg.sweep.split], cfg),
                              batch_size=cfg.train.batch_size, num_workers=cfg.train.num_workers)
        df, best = run_sweep(module, sweep_dl, cfg, dev)
        report_sweep(reporter, df, best, cfg.sweep.metric)

    onnx_files = export_for_server(cfg, task, reporter, module, df, root, out_dir) if
cfg.export.onnx.enabled else {}

    # metadata.json needs the sweep. Without it, best_checkpoint/metadata are not uploaded (ClearML still
keeps the Model).
```

```
    if df is not None:
        meta_path = out_dir / cfg.finalize.metadata_file
        write_metadata(meta_path, cfg, task, ckpt.best_model_path, module, preflight, df, best,
                       run=run, data=data, split=split, onnx=onnx_files)
        upload(task, best_checkpoint=ckpt.best_model_path, metadata=meta_path)

    # Pipeline steps have been seen to end as "completed" rather than "published"; cause not investigated.
    if task and cfg.finalize.publish_on_completion:
        task.publish_on_completion(True)
```

What it does.

Read it as the list in section 3.1:

1. Settings: `init_clearml`, a fixed seed, and the resolved config saved as an artifact.
2. Data: `resolve_root`; then the data gate, unless `preflight.enabled=false` (the pipeline sets that, because `domain_check` already ran).
3. Training: Lightning's `Trainer` with mixed precision on a GPU, the best checkpoint kept by `val/iou`, and early stopping after `train.patience` epochs without improvement. Then the best checkpoint is loaded back.
4. Sweep: every TTA x threshold on the validation split; results to ClearML and `metadata.json`.
5. ONNX: the model and its sidecar; the threshold is the best one **without TTA**, because the server does no TTA.
6. Publish: the task is set to become read-only when it completes.

6.7 Loss and metrics (`src/training/module.py`)

`src/training/module.py`, lines 15 to 18:

```
def confusion(pred, y, dims) -> torch.Tensor:
    """tp, fp, fn summed over `dims`, stacked on a new last axis."""
    tp, fp, fn = pred * y, pred * (1 - y), (1 - pred) * y
    return torch.stack([tp.sum(dims), fp.sum(dims), fn.sum(dims)], -1)
```

`src/training/module.py`, lines 21 to 23:

```
def iou_f1(counts):
    tp, fp, fn = counts.unbind(-1)
    return tp / (tp + fp + fn + 1e-8), 2 * tp / (2 * tp + fp + fn + 1e-8)
```

`src/training/module.py`, lines 50 to 58:

```
    def loss(self, logits, y):
        c = self.cfg.loss
        pos_weight = None
        if c.pos_weight is not None:            # one number, or one per class (broadcast over H, W)
```

```
            pos_weight = torch.as_tensor(c.pos_weight, dtype=torch.float32, device=logits.device)
            if pos_weight.ndim == 1:
                pos_weight = pos_weight.view(-1, 1, 1)
        bce = F.binary_cross_entropy_with_logits(logits, y, pos_weight=pos_weight)
        return c.bce_weight * bce + c.dice_weight * self.dice(logits, y)
```

`src/training/module.py`, lines 79 to 94:

```
    def on_validation_epoch_end(self):
        iou, f1 = iou_f1(self._counts)          # one value per class
        metrics = {"val/iou": iou.mean(), "val/f1": f1.mean()}
        if len(self.names) > 1:
            metrics |= {f"val/iou_{n}": v for n, v in zip(self.names, iou)}
            metrics |= {f"val/f1_{n}": v for n, v in zip(self.names, f1)}
        self.log_dict(metrics)
        if self.trainer.sanity_checking:
            return
        if metrics["val/iou"].item() > self.best_iou:
            self.best_iou, self.best_epoch = metrics["val/iou"].item(), self.current_epoch
        if isinstance(self.logger, ClearMLLogger):
            x, y, p = self._sample
            for c, name in enumerate(self.names):
                self.logger.report_image(f"val/samples_{name}", "pred(red) vs gt(green)",
                                         overlay(x, y[c:c + 1], p[c:c + 1]), self.current_epoch)
```

What it does.

- `confusion` counts true positives, false positives and false negatives; `iou_f1` turns them into IoU (`tp / (tp + fp + fn)`) and F1 (`2tp / (2tp + fp + fn)`). The sweep uses the same two functions.
- `loss` = `bce_weight x BCE + dice_weight x Dice`. BCE scores every pixel; Dice scores the overlap of the whole mask, which helps thin or rare classes such as road. `pos_weight` can make positive pixels count more.
- `on_validation_epoch_end`: logs `val/iou` (the mean over classes: this is what picks the best checkpoint) and, with several classes, `val/iou_<class>`; it also remembers the best epoch and sends one picture per class to ClearML (prediction red, truth green).

6.8 Choosing the threshold: the sweep (`src/training/sweep.py`)

`src/training/sweep.py`, lines 26 to 51:

```
@torch.no_grad()
def run_sweep(module, loader, cfg, device) -> tuple[pd.DataFrame, dict]:
    """One row per (tta, threshold): mean `iou`/`f1` over classes plus `iou_<class>`/`f1_<class>`.
    best = the row with the best mean; best["class_thresholds"] = the best threshold per class at that
tta."""
    s, names = cfg.sweep, module.names
```

```
    thr = torch.as_tensor(np.arange(s.thresholds.start, s.thresholds.stop + 1e-9, s.thresholds.step),
                          dtype=torch.float32, device=device)
    module.to(device).eval()
    rows = []
    for tta in s.tta:
        counts = torch.zeros(len(thr), len(names), 3, device=device)
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            probs = torch.stack([_inv(module(_fwd(x, k, f)).sigmoid(), k, f) for k, f in
TTA_SETS[tta]]).mean(0)
            counts += confusion((probs[None] > thr.view(-1, 1, 1, 1, 1)).float(), y, (1, 3, 4))   # (T, C,
3)
        iou, f1 = iou_f1(counts)
        for k, t in enumerate(thr.tolist()):
            row = {"tta": tta, "threshold": round(t, 4), "iou": iou[k].mean().item(), "f1":
f1[k].mean().item()}
            for c, n in enumerate(names):
                row[f"iou_{n}"], row[f"f1_{n}"] = iou[k, c].item(), f1[k, c].item()
            rows.append(row)
    df = pd.DataFrame(rows)
    best = df.loc[df[s.metric].idxmax()].to_dict()
    at_tta = df[df.tta == best["tta"]]
    best["class_thresholds"] = {n: float(at_tta.loc[at_tta[f"{s.metric}_{n}"].idxmax(), "threshold"]) for n
in names}
    return df, best
```

What it does.

For each TTA setting (`none`, `flip` = 4 views, `d4` = 8 views) the model predicts every validation tile, averaging the views after undoing each flip/rotation. One pass then scores **all thresholds at once** (0.30 to 0.70 by default): `probs[None] > thr.view(...)` gives one prediction per threshold. The result is a table with one row per (TTA, threshold); `best` is the best row, and `best["class_thresholds"]` the best threshold per class. If the best value sits at the edge of the range (road: 0.70), widen `sweep.thresholds`.

6.9 The data gate (`src/gates/preflight.py`)

`src/gates/preflight.py`, lines 21 to 36:

```
def knn_interleaving(fa: np.ndarray, fb: np.ndarray, k: int) -> dict:
    """How often a sample's k nearest neighbours come from the OTHER split, vs. what a perfect mix would
give.
    shift_score: 0 = interleaved, 1 = fully separated."""
    X = StandardScaler().fit_transform(np.vstack([fa, fb]))
    split = np.r_[np.zeros(len(fa), int), np.ones(len(fb), int)]          # 0 = reference, 1 = candidate
    k = min(k, len(X) - 1)
    neighbours = NearestNeighbors(n_neighbors=k + 1).fit(X).kneighbors(X, return_distance=False)[:, 1:]  #
drop itself
```

```
    other = (split[neighbours] != split[:, None]).mean(1)                # share of neighbours from the
other split
    expected = np.where(split == 0, len(fb), len(fa)) / (len(X) - 1)       # that share if the splits were
mixed

    def shift(rows) -> float:
        return float(np.clip(1 - other[rows].mean() / expected[rows].mean(), 0, 1))

    return {"other_domain_neighbour_frac": float(other.mean()), "expected_if_mixed": float(expected.mean()),
            "shift_score": shift(slice(None)), "shift_score_ref": shift(split == 0),
            "shift_score_cand": shift(split == 1), "k": int(k)}
```

`src/gates/preflight.py`, lines 115 to 123:

```
def gate_preflight(cfg, task, reporter, root: Path, out_dir: Path) -> dict:
    """Sample both splits and run the checks; a violated `gate` fails the Task and exits with code 2."""
    p, sp = cfg.preflight, cfg.data.splits
    ref, cand = (TileDataset(root, sp[s], cfg, limit=p.max_samples, seed=cfg.seed)
                 for s in (p.reference_split, p.candidate_split))
    try:
        return run_preflight(cfg, reporter, ref, cand, out_dir)
    except PreflightFailed as e:
        fail_gate(task, "preflight_gate", str(e))
```

What it does.

- `knn_interleaving`: every image is summarised as a few numbers (per-channel mean, std, percentiles). For each image, look at its 10 nearest neighbours: if train and val are alike, about as many neighbours come from the other split as chance predicts. `shift_score` 0 = well mixed, 1 = the two splits look completely different. The gate fails above `max_shift` (0.5); measured on LoveDA: 0.004.
- `gate_preflight`: samples up to `max_samples` tiles of each split, runs the checks and, if a check in `gate` mode fails, calls `fail_gate` (exit code 2). Used by `domain_check` and at the start of training.

6.10 The serving measurement (`src/gates/serve_check.py`)

`src/gates/serve_check.py`, lines 30 to 49:

```
class Model:
    def __init__(self, path: Path, intra: int):
        o = ort.SessionOptions()
        o.enable_mem_pattern, o.enable_cpu_mem_arena = True, False      # as in the server: memory goes back
at once
        o.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
        o.intra_op_num_threads, o.inter_op_num_threads = intra, 1
        self.name = path.stem
        self.session = ort.InferenceSession(str(path), sess_options=o, providers=["CPUExecutionProvider"])
```

```
        self.input = self.session.get_inputs()[0].name
        self.outputs = [x.name for x in self.session.get_outputs()]
        side = path.with_suffix(".onnx.json")
        self.threshold = (json.loads(side.read_text()).get("threshold") if side.exists() else None) or 0.5
        self.lock, self.seconds = threading.Lock(), []

    def predict(self, tensor: np.ndarray, size: int):
        with self.lock:                                          # one run at a time per model, like
the server
            t0 = time.perf_counter()
            out = self.session.run(self.outputs, {self.input: tensor})
            self.seconds.append(time.perf_counter() - t0)
        return postprocess_semantic_masks(out, (size, size), 1.0, (0.0, 0.0), size, self.threshold)
```

`src/gates/serve_check.py`, lines 52 to 67:

```
def request(models, tiles: int, size: int, pool, max_inflight: int) -> float:
    """One request = `tiles` tiles x every model, submitted with the server's in-flight limit."""
    rng = np.random.default_rng(0)
    t0, inflight = time.perf_counter(), set()
    for _ in range(tiles):
        tile = rng.integers(0, 255, (size, size, 3), dtype=np.uint8)                    # BGR like cv2
        tensor = np.ascontiguousarray(tile[:, :, ::-1].astype(np.float32).transpose(2, 0, 1)[None] / 255.0)
        for m in models:
            inflight.add(pool.submit(m.predict, tensor, size))
            if len(inflight) >= max_inflight:
                done, inflight = cf.wait(inflight, return_when=cf.FIRST_COMPLETED)
                for f in done:
                    f.result()
    for f in cf.as_completed(inflight):
        f.result()
    return time.perf_counter() - t0
```

What it does.

This copies how the Chisel server runs its models, so the numbers mean the same thing:

- `Model`: an onnxruntime CPU session with the server's options (2 threads per model, memory arena off), the threshold from the sidecar, and a **lock**: one model runs one tile at a time, as in the server.
- `request`: one user request = every tile x every model, submitted to a thread pool of `--workers` (the server's `THREAD_POOL_SIZE`, 2), with at most `max(2 x workers, models)` jobs waiting. The wall time of this is the latency.

6.11 The serving gate as a task (`src/gates/serve_gate.py`)

`src/gates/serve_gate.py`, lines 68 to 76:

```
def measure_isolated(model_dir: Path, s, out_json: Path) -> dict:
    """Measure in a FRESH process: the export / download above would otherwise count in the peak memory."""
    args = {"models": model_dir, "tiles": s.tiles, "size": s.tile_size, "repeats": s.repeats, "workers":
s.workers,
            "intra": s.intra, "cpu-limit": s.cpu_limit, "memory-limit-gib": s.memory_limit_gib,
            "max-request-s": s.max_request_s, "mode": "diagnostic", "out": out_json}
    cmd = [sys.executable, "-m", "src.gates.serve_check"]
    cmd += [x for k, v in args.items() if v is not None for x in (f"--{k}", str(v))]
    subprocess.run(cmd, check=True, cwd=REPO)
    return json.loads(out_json.read_text())
```

`src/gates/serve_gate.py`, lines 79 to 105:

```
@hydra.main(config_path="../../conf", config_name="config", version_base="1.3")
def main(cfg: DictConfig):
    cfg.clearml.task_name = "serve-gate"                        # stable name: the pipeline finds the base
task by it
    faulthandler.dump_traceback_later(300, repeat=True)         # a stall prints its stack into the Task
console
    out_dir = Path(HydraConfig.get().runtime.output_dir)
    task, cfg = init_clearml(cfg)
    reporter, s = Reporter(task), cfg.serve

    workdir = Path(s.folder) if s.source == "folder" else Path(tempfile.mkdtemp(prefix="serve_gate_"))
    files = {"random": lambda: models_from_spec(s.spec, workdir, s.tile_size),
             "tasks": lambda: models_from_pipeline(task, workdir),
             "folder": lambda: sorted(workdir.glob("*.onnx"))}[s.source]()
    print(f"[serve] source={s.source}: {[f.stem for f in files]}", flush=True)

    res = {**measure_isolated(workdir, s, out_dir / "serve_check.json"), "source": s.source}
    for key in ("request_mean_s", "request_p95_s", "peak_rss_gib", "cpu_cores_busy", "threads_in_use"):
        reporter.scalar("serve", key, res[key], 0)
        reporter.single(f"serve/{key}", res[key])
    for name, sec in res["per_model_mean_s"].items():
        reporter.scalar("serve/per_model_s", name, sec, 0)
    reporter.scalar("serve", "passed", float(res["passed"]), 0)
    (out_dir / "serve_gate.json").write_text(json.dumps(res, indent=2))
    if task:
        task.upload_artifact("serve_gate", artifact_object=res)
    print(json.dumps(res, indent=2))
    if not res["passed"] and s.mode == "gate":
        fail_gate(task, "serve_gate", "; ".join(res["violations"]))
```

What it does.

- `main` gets the models (`serve.source`: `random` exports untrained ones before training, `tasks` downloads the ONNX files of this pipeline's train steps, `folder` uses local files), measures, reports to ClearML, and fails the gate on a violation.
- `measure_isolated` runs the measurement in a **new process**. Exporting models uses a lot of memory; measured in the same process, that would count as the models' peak.

6.12 The pipeline (`src/orchestration/pipeline.py`)

`src/orchestration/pipeline.py`, lines 29 to 47:

```
def plan(c, classes: list[str] | None = None) -> list[dict]:
    """The pipeline as plain data: one dict of add_step() arguments per step, in dependency order."""
    names = classes or list(c.classes)
    if unknown := [n for n in names if n not in c.classes]:
        raise SystemExit(f"unknown classes {unknown}; pipeline.yaml has {list(c.classes)}")
    spec = ",".join(f"{n}={c.classes[n].architecture}:{c.classes[n].encoder}" for n in names)

    def step(name, kind, parents=(), extra=None):
        return {"name": name, "parents": list(parents), "base_task_project": c.project,
                "base_task_name": c.steps[kind].task, "execution_queue": c.steps[kind].queue,
                "parameter_override": overrides(c.params, extra or {})}

    trains = [step(f"train_{n}", "train", ["domain_check", "serve_gate_pre"],
                   {**c.train_params, "data": n, "architecture": c.classes[n].architecture,
                    "encoder": c.classes[n].encoder, "loss": c.classes[n].get("loss")}) for n in names]
    return [step("domain_check", "domain_check", extra={"data": "loveda"}),
            step("serve_gate_pre", "serve_gate", extra={**c.serve_params, "serve.source": "random",
"serve.spec": spec}),
            *trains,
            step("serve_gate_post", "serve_gate", [t["name"] for t in trains], {**c.serve_params,
"serve.source": "tasks"})]
```

`src/orchestration/pipeline.py`, lines 50 to 59:

```
def newest_base_task_id(project: str, name: str) -> str:
    """The most recently CREATED non-archived task with this exact name, whatever its status.
    PipelineController's own lookup by name also returns archived tasks and skips `failed` ones, so after
    re-registering it can silently pick an old base task pinned to an old commit. Creation time, not
last_update:
    stopping or editing an old base task bumps its last_update and made it look newest."""
    from clearml import Task

    if not (tasks := Task.get_tasks(project_name=project, task_name=f"^{name}$", allow_archived=False)):
        raise SystemExit(f"no non-archived base task named {name!r} in project {project!r}")
    return max(tasks, key=lambda t: t.data.created).id
```

`src/orchestration/pipeline.py`, lines 62 to 69:

```
def build(c, classes: list[str] | None = None):
    from clearml import PipelineController

    pipe = PipelineController(name=c.name, project=c.project, version=c.version, add_pipeline_tags=True)
    for step in plan(c, classes):
        project, name = step.pop("base_task_project"), step.pop("base_task_name")   # add_step takes the id,
not these
        pipe.add_step(**step, base_task_id=newest_base_task_id(project, name))
    return pipe
```

What it does.

- `plan` returns the pipeline as a plain list (this is what `--dry-run` prints): two gates, one `train_<class>` step per class after both gates, and `serve_gate_post` after all training steps. Each step carries the settings it writes into its clone (`parameter_override`), e.g. `Overrides/data=road`.
- `newest_base_task_id` picks the newest **non-archived** base task by exact name. ClearML's own lookup also returns archived tasks and skips `failed` ones, which once made a pipeline run old code.
- `build` hands the steps to ClearML's `PipelineController`.

6.13 ONNX export (`src/export_onnx.py`)

`src/export_onnx.py`, lines 49 to 55:

```
def threshold_from_rows(rows: list[dict], metric: str, class_name: str) -> float | None:
    """Best threshold among the sweep rows with tta == "none" (the server runs no TTA)."""
    rows = [r for r in rows if r["tta"] == "none"]
    if not rows:
        return None
    key = f"{metric}_{class_name}" if f"{metric}_{class_name}" in rows[0] else metric
    return float(max(rows, key=lambda r: r[key])["threshold"])
```

`src/export_onnx.py`, lines 65 to 88:

```
def export_model(network: torch.nn.Module, out: Path, names: list[str], size: int = 1280, opset: int = 17,
                 threshold: float | None = None, architecture: str = "") -> dict:
    """Wrap `network` (logits) with a sigmoid, export to ONNX, check onnxruntime == PyTorch, write the
sidecar json."""
    import onnxruntime as ort

    model = torch.nn.Sequential(network.cpu().eval(), torch.nn.Sigmoid()).eval()
    out = Path(out)
    out.parent.mkdir(parents=True, exist_ok=True)
    dummy = torch.rand(1, 3, size, size)
    torch.onnx.export(model, dummy, str(out), input_names=["images"], output_names=["probs"],
```

```
                    opset_version=opset, dynamo=False)
    sess = ort.InferenceSession(str(out), providers=["CPUExecutionProvider"])
    with torch.no_grad():
        ref = model(dummy).numpy()
    got = sess.run(["probs"], {"images": dummy.numpy()})[0]
    diff = float(np.abs(ref - got).max())
    print(f"[export] {out.name} ({out.stat().st_size / 1e6:.1f} MB) output {got.shape} max |torch - onnx| =
{diff:.2e}")
    if diff > 1e-3:
        raise RuntimeError(f"ONNX parity failed: {diff}")
    side = {"classes": names, "input_size": size, "input": "RGB float32 0..1 letterboxed, pad 114",
            "output": "sigmoid probabilities (1, C, S, S)", "tta": "none", "threshold": threshold,
            "architecture": architecture, "opset": opset}
    out.with_suffix(".onnx.json").write_text(json.dumps(side, indent=2))
    return side
```

What it does.

- `threshold_from_rows`: from the sweep table, keep only the rows with TTA `none` and take the best threshold.
- `export_model`: adds a sigmoid to the network (the file outputs probabilities), exports with a fixed input of `(1, 3, size, size)` (1280, the server's input size), then **runs the file with onnxruntime and compares it with PyTorch** on the same random input; a difference above 1e-3 stops the run. Last, it writes the sidecar `<class>.onnx.json` with the class name and the threshold.

6.14 Making the file smaller (`src/shrink.py`)

`src/shrink.py`, lines 84 to 153:

```
def quantize_tensor(name, value_tensor, original_output_tensor_name, root_graph):
    """Quantize a constant tensor to int8, reconstructed via Cast/Mul/Add.

    Args:
        name: The name of the tensor to quantize.
        value_tensor: The tensor to quantize.
        original_output_tensor_name: The name of the original tensor in the graph.
        root_graph: The root graph of the model.
    """
    dtype = value_tensor.dtype
    np_dtype = onnx.helper.np_dtype_to_tensor_dtype(dtype)
    float_values = value_tensor.values
    min_val = np.min(float_values)
    max_val = np.max(float_values)
    range_val = max_val - min_val
    inverse_range = 1.0 / range_val
    zero_point = round(-min_val * inverse_range * 255.0) - 128
    quantized_values = np.round(float_values * inverse_range * 255.0) + zero_point
```

```
    quantized_values = np.clip(quantized_values, -128, 127).astype(np.int8)

    quantized_tensor = gs.Constant(name=f"{name}_quantized", values=quantized_values)

    scale_value = range_val / 255.0
    scale_tensor = gs.Constant(
        name=f"{name}_scale", values=np.array([scale_value], dtype=dtype)
    )

    zero_point_tensor = gs.Constant(
        name=f"{name}_zero_point",
        values=np.array([-zero_point * scale_value], dtype=dtype),
    )

    # DequantizeLinear is surprisingly slow in OnnxRuntime, so achieve the
    # same effect with a Cast, Mul, and Add.
    cast_tensor = gs.Variable(
        name=f"{name}_cast_tensor", dtype=dtype, shape=value_tensor.shape
    )
    cast_node = gs.Node(
        op="Cast",
        name=f"{name}_cast_node",
        inputs=[quantized_tensor],
        outputs=[cast_tensor],
        attrs={"to": np_dtype},
    )

    mul_tensor = gs.Variable(
        name=f"{name}_mul_tensor", dtype=dtype, shape=value_tensor.shape
    )
    mul_node = gs.Node(
        op="Mul",
        name=f"{name}_mul_node",
        inputs=[cast_tensor, scale_tensor],
        outputs=[mul_tensor],
    )

    add_tensor = gs.Variable(
        name=f"{name}_add_tensor", dtype=dtype, shape=value_tensor.shape
    )
    add_node = gs.Node(
        op="Add",
        name=f"{name}_add_node",
        inputs=[mul_tensor, zero_point_tensor],
        outputs=[add_tensor],
    )
```

```
    replace_tensor_for_subgraph(root_graph, original_output_tensor_name, add_tensor)

    root_graph.nodes.append(cast_node)
    root_graph.nodes.append(mul_node)
    root_graph.nodes.append(add_node)
```

What it does.

For every large weight tensor: find its min and max, store it as **int8** (256 levels) plus a scale and an offset, and add three small operations (Cast, Mul, Add) that turn it back into float when the model runs. The file becomes about 4x smaller (97.7 -> 24.6 MB); latency was unchanged in the test. The code uses Cast/Mul/Add instead of ONNX's `DequantizeLinear` because the latter is slow in onnxruntime (comment in the original code). The accuracy of a shrunk *trained* model is not measured yet.

6.15 Building the network (`src/model_factory.py`)

`src/model_factory.py`, lines 10 to 21:

```
def build_model(cfg: DictConfig) -> nn.Module:
    """cfg.model = {_target_: smp.create_model, arch, encoder_name, encoder_weights, in_channels, classes,
...}.
    `pretrain.source` smp: create_model already loaded the weights; local / torchgeo: they replace the
encoder's."""
    model = instantiate(cfg.model, _convert_="all") # construct the model using Hydra's instantiate and
convert all to plain python so smp can understand
    pre = cfg.get("pretrain") # get the pretrain configuration if it exists example unet
    source = pre.source if pre is not None else None
    if source == "local":       # offline-safe: a file made by `python -m src.export_weights`
        state = torch.load(pre.path, map_location="cpu", weights_only=True)
        _load_encoder(model.encoder, state, f"local weights {pre.path}")
    elif source == "torchgeo":
        _load_encoder(model.encoder, _torchgeo_state(pre, cfg.model.in_channels), f"torchgeo
{pre.weights_name}")
    return model
```

What it does.

`instantiate(cfg.model)` calls `segmentation_models_pytorch.create_model` with the values from `conf/architecture/` and `conf/encoder/` (e.g. `arch=Unet, encoder_name=resnet34, encoder_weights=imagenet`). Optional extra step: load encoder weights from a local file (offline) or from torchgeo, and fail if keys are missing.

6.16 Inside the server: probabilities -> mask (`serving/postprocess_semantic.py`)

`serving/postprocess_semantic.py`, lines 19 to 39:

```python
def postprocess_semantic_masks(
    outputs: List[np.ndarray],
    original_shape: Tuple[int, int],
    ratio: float,
    pad: Tuple[float, float],
    input_size: int = 1280,
    conf_threshold: float = 0.5,
    use_bbox_cropping: bool = False,
) -> Optional[np.ndarray]:
    if not outputs or outputs[0].size == 0:
        return None
    probs = outputs[0][0, 0]                                    # (S, S): first (only) class channel

    dw, dh = pad
    top, left = max(0, int(round(dh - 0.1))), max(0, int(round(dw - 0.1)))
    bottom = min(input_size, top + int(round(original_shape[0] * ratio)))
    right = min(input_size, left + int(round(original_shape[1] * ratio)))

    tile = cv2.resize(probs[top:bottom, left:right], (original_shape[1], original_shape[0]),
                      interpolation=cv2.INTER_LINEAR)
    return (tile > conf_threshold).astype(np.uint8) * 255
```

What it does.

The server shrinks or enlarges every tile to fit 1280 x 1280 and pads the rest grey ("letterbox"). This undoes that: cut away the padding (`top`, `left`, `bottom`, `right` from the server's own `pad` and `ratio`), resize the probabilities back to the tile's real size, and threshold. The result is the same 0/255 mask the server's YOLO code returns, so nothing else in the server changes.

6.17 The server change (`serving/chisel_server.patch`)

`serving/chisel_server.patch`, lines 1 to 57:

```
--- a/server.py
+++ b/server.py
@@ -19,6 +19,7 @@
 from libs.mask_processor import MaskProcessor
 from libs.preprocess import preprocess_image
 from libs.postprocess import postprocess_yolo_masks
+from libs.postprocess_semantic import postprocess_semantic_masks
 from libs.slice_inference import slice_and_infer

 # config
@@ -89,6 +90,14 @@
         self.output_names = None
         self._lock = threading.Lock()
```

```
          self.logger = logging.getLogger(__name__)
+         # semantic models come with a <name>.onnx.json sidecar that holds their threshold (best at TTA
"none")
+         self.threshold = None
+         sidecar = model_path + ".json"
+         if os.path.exists(sidecar):
+             with open(sidecar) as f:
+                 self.threshold = json.load(f).get("threshold")
+             if self.threshold is None:
+                 raise ValueError(f"{sidecar} has no threshold")
          self._setup_session()

      def _setup_session(self):
@@ -245,15 +254,22 @@
              if not outputs:
                  return category, None

-             mask = postprocess_yolo_masks(
-                 outputs,
-                 original_shape,
-                 ratio,
-                 pad,
-                 input_size=input_size,
-                 conf_threshold=confidence,
-                 use_bbox_cropping=use_bbox_cropping,
-             )
+             model = self.models[category]
+             if model.threshold is not None:        # semantic model: its own sidecar threshold, not the
YOLO confidence
+                 mask = postprocess_semantic_masks(
+                     outputs, original_shape, ratio, pad,
+                     input_size=input_size, conf_threshold=model.threshold,
+                 )
+             else:
+                 mask = postprocess_yolo_masks(
+                     outputs,
+                     original_shape,
+                     ratio,
+                     pad,
+                     input_size=input_size,
+                     conf_threshold=confidence,
+                     use_bbox_cropping=use_bbox_cropping,
+                 )
              return category, mask

          except Exception as e:
```

What it does.

Two changes to `server.py`. When the server loads a model, it looks for `<model>.onnx.json`; if there is one, its `threshold` is kept. When it predicts, a model with a threshold goes through `postprocess_semantic_masks`; a model without one (the current YOLO files) goes through the old code with its old `confidence`. So old and new models can even be mixed.

6.18 Turning a Kaggle notebook into an agent (`kaggle/start_agent.py`)

`kaggle/start_agent.py`, lines 25 to 41:

```
secrets = UserSecretsClient()
os.environ["CLEARML_API_HOST"] = "https://api.clear.ml"        # hosted server; change for a self-hosted one
os.environ["CLEARML_WEB_HOST"] = "https://app.clear.ml"
os.environ["CLEARML_FILES_HOST"] = "https://files.clear.ml"
os.environ["CLEARML_API_ACCESS_KEY"] = secrets.get_secret("CLEARML_KEY").strip()
os.environ["CLEARML_API_SECRET_KEY"] = secrets.get_secret("CLEARML_SECRET").strip()
try:                                                  # private repo: the agent clones it with these
    os.environ["CLEARML_AGENT_GIT_USER"] = secrets.get_secret("GIT_USER")
    os.environ["CLEARML_AGENT_GIT_PASS"] = secrets.get_secret("GIT_TOKEN")
except Exception:
    print("no GIT_USER / GIT_TOKEN secret: assuming a public repo")

# Kaggle already ships a CUDA build of torch: let the agent's venv reuse it instead of reinstalling.
Path("~/clearml.conf").expanduser().write_text("agent {\n  package_manager { system_site_packages: true
}\n}\n")

# Install and start the ClearML agent with the specified queue and GPU.
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "clearml-agent"])
```

What it does.

Reads your ClearML keys from Kaggle's secrets, points the agent at app.clear.ml, tells it to reuse Kaggle's preinstalled CUDA torch instead of installing its own (`system_site_packages`), installs `clearml-agent`, and starts it on the `kaggle` queue with GPU 0. The last line keeps running; the agent stops when the notebook session ends.

6.19 Which settings ClearML may change (`conf/config.yaml`)

`conf/config.yaml`, lines 72 to 95:

```
  tunable:
    - architecture
    - encoder
    - loss                    # config group: loss/<class>.yaml (weights per class), default = 0.5 BCE + 0.5
Dice
    - data                    # config group: which class / dataset config (building, road, water,
```

```
vegetation, loveda)
    - optimizer.lr
    - optimizer.weight_decay
    - loss.dice_weight
    - loss.bce_weight
    - train.batch_size
    - train.epochs
    - augment.hflip
    - augment.vflip
    - augment.rot90
    - data.root
    - data.clearml_dataset_id
    - preflight.enabled
    - preflight.domain_shift.mode
    - preflight.label_balance.mode
    - serve.source
    - serve.spec
    - serve.memory_limit_gib
    - serve.max_request_s
    - serve.tiles
```

What it does.

Only these keys can be changed from the ClearML UI, a pipeline or HPO (they appear as "Overrides/" on a task). Anything else stays as the YAML and the command line say. To open a new knob: add its key here, push, and re-register the base tasks.

6.20 One architecture per class (`pipeline/pipeline.yaml`)

`pipeline/pipeline.yaml`, lines 25 to 29:

```
classes:
  building:   {architecture: unet, encoder: resnet34_imagenet, loss: building}
  road:       {architecture: unet, encoder: resnet34_imagenet, loss: road}
  water:      {architecture: unet, encoder: resnet34_imagenet, loss: water}
  vegetation: {architecture: unet, encoder: resnet34_imagenet, loss: vegetation}
```

What it does.

The single table the pipeline reads for the classes. The names must match `conf/data/<class>.yaml`; the values name files in `conf/architecture/` and `conf/encoder/`. Change a line (e.g. `road: {architecture: fpn, ...}`) and both the pre-training gate and that class's training step use it.